FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

INFRASTRUCTURE DEVOPS

RAPPORT

PRÉSENTÉ PAR :
KARROUM MAROUA
BADR MARTAJ
NAFII MOHAMED
MOURAFIQ MOUAD

ENCADRÉ PAR :
MADAME BENABOU

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet. Tout d'abord, je remercie vivement mon encadrant pour ses conseils avisés, sa disponibilité et son soutien constant tout au long du travail, ainsi que pour m'avoir guidée dans l'application des bonnes pratiques DevOps et cloud-native.

Je remercie également mes collègues et amis pour leurs suggestions constructives, leur aide lors des tests et leur motivation qui ont rendu ce projet plus enrichissant et collaboratif. Un grand merci aux formateurs et aux ressources pédagogiques qui m'ont permis de renforcer mes compétences techniques, notamment en Docker, Kubernetes, Terraform, Ansible et CI/CD.

Enfin, je tiens à remercier ma famille pour leur patience, leur encouragement et leur soutien moral tout au long de ce projet. Sans toutes ces contributions, la réalisation de ce travail n'aurait pas été possible.

Résumé

Ce projet avait pour objectif de concevoir et déployer une solution DevOps complète pour l'application Expense Splitter, permettant d'assurer un déploiement automatisé, fiable, sécurisé et reproductible sur le cloud. L'application Node.js/Express, accompagnée d'une base de données MongoDB, a été conteneurisée avec Docker pour garantir portabilité, isolation et cohérence entre les environnements de développement et de production. L'infrastructure cloud a été provisionnée automatiquement sur Microsoft Azure via Terraform, incluant la création des machines virtuelles, des réseaux et des règles de sécurité, tandis qu'Ansible a été utilisé pour configurer le cluster Kubernetes K3s et automatiser le déploiement des conteneurs. Une chaîne CI/CD complète a été mise en place avec GitHub Actions pour automatiser les tests, la construction, le push des images Docker et le déploiement sur le cluster, garantissant ainsi des livraisons rapides et fiables. Parallèlement, une stack de monitoring avancée intégrant Prometheus, Grafana, Loki et Promtail a été déployée pour superviser l'état du cluster, collecter les métriques et centraliser les logs applicatifs, offrant une observabilité complète et facilitant la détection des anomalies. Ce projet a permis d'acquérir des compétences pratiques en conteneurisation, orchestration Kubernetes, Infrastructure as Code et automatisation DevOps, tout en démontrant l'efficacité de l'intégration continue et du déploiement continu dans un environnement cloud moderne.

Abstract

The purpose of this project was to design and deploy a comprehensive DevOps solution for the Expense Splitter application, ensuring automated, reliable, secure, and reproducible cloud deployment. The Node.js/Express application, coupled with a MongoDB database, was containerized with Docker to ensure portability, isolation, and consistency across development and production environments. The cloud infrastructure was automatically provisioned on Microsoft Azure using Terraform, including virtual machines, networks, and security rules, while Ansible was employed to configure the K3s Kubernetes cluster and automate container deployment. A complete CI/CD pipeline was established with GitHub Actions to automate testing, Docker image building, pushing, and deployment to the cluster, ensuring fast and reliable releases. Simultaneously, an advanced monitoring stack including Prometheus, Grafana, Loki, and Promtail was deployed to supervise cluster health, collect metrics, and centralize application logs, providing full observability and facilitating anomaly detection. This project allowed the acquisition of practical skills in containerization, Kubernetes orchestration, Infrastructure as Code, and DevOps automation, while demonstrating the effectiveness of continuous integration and continuous deployment in a modern cloud environment.

Introduction

Avec l'évolution rapide des technologies cloud et des pratiques DevOps, les entreprises cherchent aujourd'hui à déployer des applications de manière rapide, fiable et sécurisée, tout en garantissant leur scalabilité et leur maintenabilité. Dans ce contexte, l'utilisation de conteneurs orchestrés par Kubernetes, combinée à une approche Infrastructure as Code (IaC) et à des pipelines CI/CD automatisés, permet de réduire considérablement le temps de mise en production tout en minimisant les erreurs humaines. Le projet présenté dans ce rapport consiste à déployer l'application *Expense Splitter*, une application de gestion de dépenses en groupe développée en Node.js/Express avec une base de données MongoDB, dans un environnement cloud basé sur Microsoft Azure, en appliquant les bonnes pratiques DevOps afin d'automatiser l'ensemble du cycle de vie de l'application, depuis la phase de développement jusqu'au monitoring en production. Toutefois, le déploiement d'une application cloud-native présente plusieurs défis majeurs, notamment la garantie de la fiabilité et de la disponibilité du service même lors des mises à jour ou des pannes, la nécessité d'assurer la scalabilité face à l'augmentation du nombre d'utilisateurs, la réduction de l'intervention humaine grâce à l'automatisation du déploiement et de la configuration de l'infrastructure, ainsi que la mise en place d'une observabilité complète des métriques système, de l'état des pods Kubernetes et des logs applicatifs pour détecter rapidement toute anomalie, sans oublier les exigences liées à la sécurité telles que la gestion des secrets, des accès SSH et de la configuration réseau sur le cloud. Afin de répondre à ces enjeux, ce projet vise à mettre en place une solution DevOps end-to-end intégrant le développement et les tests locaux de l'application, sa conteneurisation avec Docker**,** le provisioning automatique de l'infrastructure Azure à l'aide de Terraform, la configuration d'un cluster K3s Kubernetes via Ansible, la mise en place d'une chaîne CI/CD automatisée avec GitHub Actions, ainsi que l'installation d'une stack complète de monitoring composée de Prometheus, Grafana, Loki et Promtail pour la collecte des métriques et la centralisation des logs, tout en assurant une

documentation claire permettant la reproduction complète de l’infrastructure et du processus de déploiement dans un environnement professionnel fiable, maintenable et reproductible.

CHAPITRE 1 : Architecture globale et environnement technique DevOps

1.1. Introduction

Ce chapitre présente l'architecture globale mise en place pour assurer le déploiement automatisé de l'application *Expense Splitter* dans un environnement cloud. Cette architecture repose sur une approche DevOps intégrant la conteneurisation, l'orchestration Kubernetes, l'Infrastructure as Code et une chaîne CI/CD afin de garantir un déploiement fiable, reproductible et maintenable.

1.2. Présentation générale de l'architecture DevOps

L'architecture du projet repose sur une approche cloud-native combinant Docker pour la conteneurisation, Kubernetes (K3s) pour l'orchestration, Terraform pour le provisioning de l'infrastructure cloud sur Microsoft Azure, Ansible pour la configuration des machines virtuelles et GitHub Actions pour l'automatisation du pipeline CI/CD. Une stack de monitoring composée de Prometheus, Grafana, Loki et Promtail assure la supervision du cluster et la centralisation des logs applicatifs.

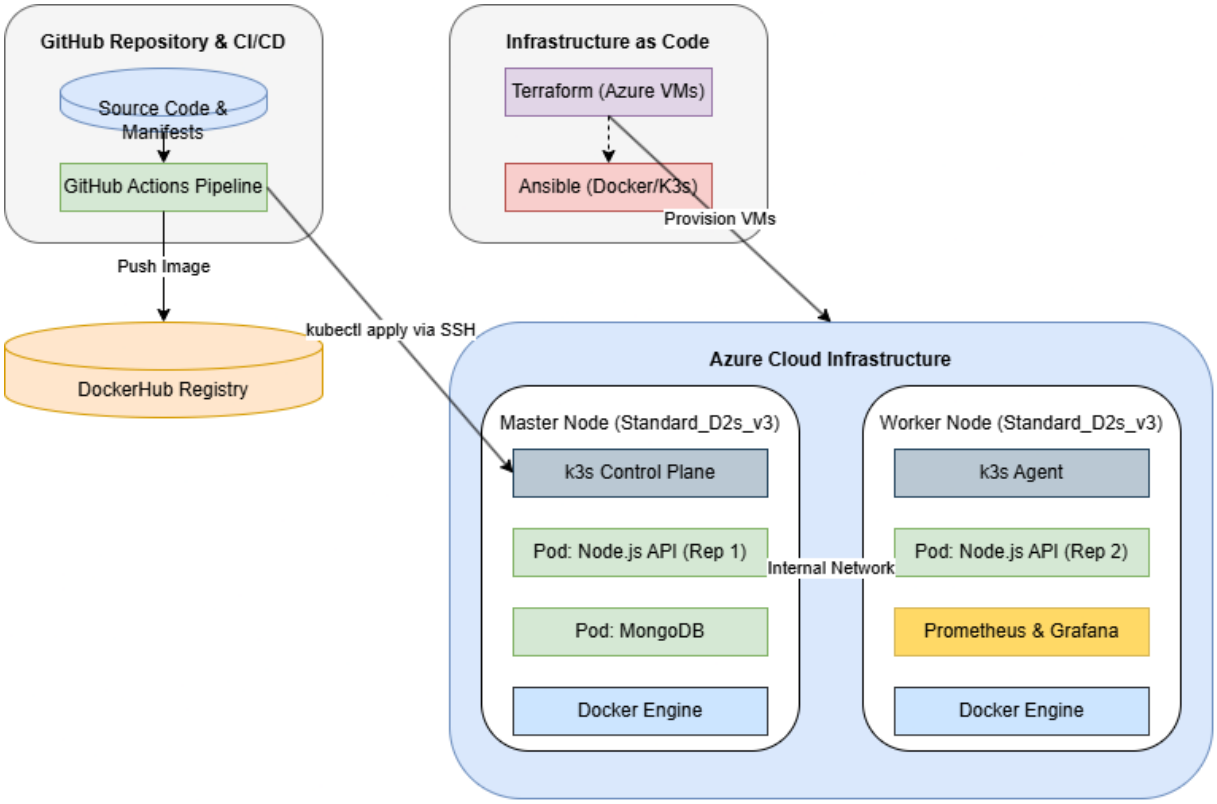

Figure 1: Architecture globale de la solution DevOps

1.3. Description du workflow DevOps

L'architecture mise en place repose sur un workflow DevOps automatisé permettant d'assurer l'intégration continue et le déploiement continu de l'application.

Le cycle de déploiement commence par le *push* du code source sur GitHub, ce qui déclenche automatiquement le pipeline GitHub Actions chargé d'exécuter les différentes étapes du processus de déploiement. Ce pipeline construit l'image Docker de l'application puis la publie sur Docker Hub.

Ensuite, les fichiers de configuration Kubernetes sont utilisés pour déployer automatiquement l'application sur le cluster K3s hébergé sur l'infrastructure Azure. L'infrastructure elle-même est provisionnée automatiquement grâce à Terraform, tandis que Ansible assure la configuration des machines virtuelles et l'installation du cluster Kubernetes.

Enfin, une stack de monitoring basée sur Prometheus, Grafana, Loki et Promtail est déployée via Helm, permettant de superviser l'état du cluster et de centraliser les logs applicatifs.

1.4. Infrastructure cloud utilisée

L'infrastructure cloud utilisée dans ce projet repose sur la plateforme Microsoft Azure, qui fournit un environnement flexible et scalable pour le déploiement d'applications conteneurisées.

Le provisioning de l'infrastructure est réalisé à l'aide de Terraform, permettant la création automatique de deux machines virtuelles Linux :

- une machine configurée comme nœud master Kubernetes
- une machine configurée comme nœud worker

Un Network Security Group (NSG) est également configuré afin d'autoriser les ports nécessaires au fonctionnement du cluster :

- Port **22** : accès SSH
- Port **80** : accès HTTP
- Port **6443** : API Kubernetes
- Port **30080** : accès à l'application via NodePort

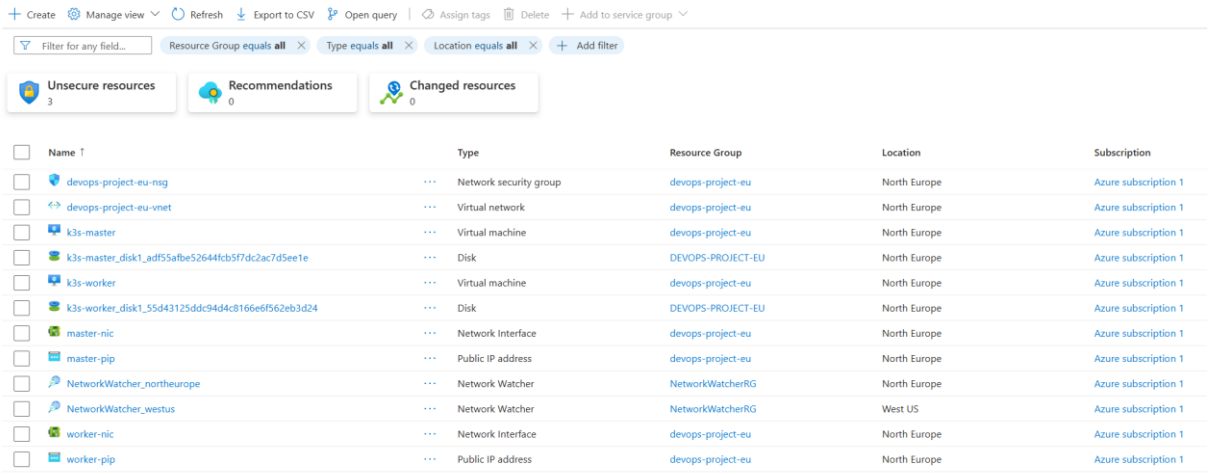

Figure 2: Ressources déployées sur Microsoft Azure

Cette configuration garantit la communication entre les différents composants du cluster tout en assurant un niveau de sécurité adapté.

1.5. Description des composants

Le projet utilise GitHub pour la gestion du code source, GitHub Actions pour la CI/CD, Docker pour la conteneurisation, Docker Hub pour le stockage des images, Terraform pour le provisioning de l'infrastructure, Ansible pour la configuration des machines, K3s pour l'orchestration des conteneurs et Helm pour le déploiement des services de monitoring. La supervision est assurée par Prometheus, Grafana, Loki et Promtail.

1.6. Justification des choix technologiques

Le choix de Microsoft Azure s'explique par sa simplicité d'utilisation et sa compatibilité avec les outils DevOps. Terraform permet une gestion automatisée et reproductible de l'infrastructure, tandis que Ansible facilite la configuration des machines virtuelles grâce à son approche centralisée. K3s a été choisi pour sa légèreté et sa simplicité d'installation. Enfin, GitHub Actions automatise efficacement le pipeline CI/CD, et la stack Prometheus–Grafana–Loki–Promtail assure une observabilité complète du système.

1.7. Conclusion :

Dans ce chapitre, nous avons présenté l'architecture globale du projet ainsi que l'environnement technique utilisé pour assurer le déploiement automatisé de l'application *Expense Splitter* sur une infrastructure cloud Azure. Nous avons également décrit les

principaux composants du système ainsi que les outils DevOps employés pour garantir une gestion efficace, sécurisée et reproductible de l'infrastructure. Le chapitre suivant sera consacré à la présentation détaillée de l'application et à sa conteneurisation avec Docker.

Chapitre 2 : Développement, conteneurisation et Infrastructure as Code

2.1. Présentation de l'application

L'application Expense Splitter est un outil de gestion de dépenses en groupe, développé en Node.js/Express avec une base de données MongoDB. Elle permet de gérer des utilisateurs et des groupes, d'enregistrer des dépenses et de calculer automatiquement les balances entre les membres. L'objectif principal est de déployer cette application dans un environnement cloud automatisé, scalable et sécurisé.

2.2. Architecture logicielle

L'architecture logicielle repose sur un modèle API RESTful avec Express.js et une base de données MongoDB. L'application est conteneurisée avec Docker, ce qui permet de garantir la portabilité et l'isolation de l'environnement d'exécution. Les images Docker sont stockées sur Docker Hub pour faciliter leur déploiement sur le cluster Kubernetes.

2.3. Fonctionnalités principales

- Gestion des utilisateurs et des groupes
- Création et modification des dépenses
- Calcul automatique des soldes et balances
- Health check via /api/health pour surveiller l'état de l'API

2.4. Tests locaux

Avant la conteneurisation, l'application a été testée localement afin de vérifier la connectivité avec MongoDB, la validité des routes API et la stabilité du serveur Node.js. Ces tests garantissent le bon fonctionnement de l'application avant son déploiement sur le cloud.

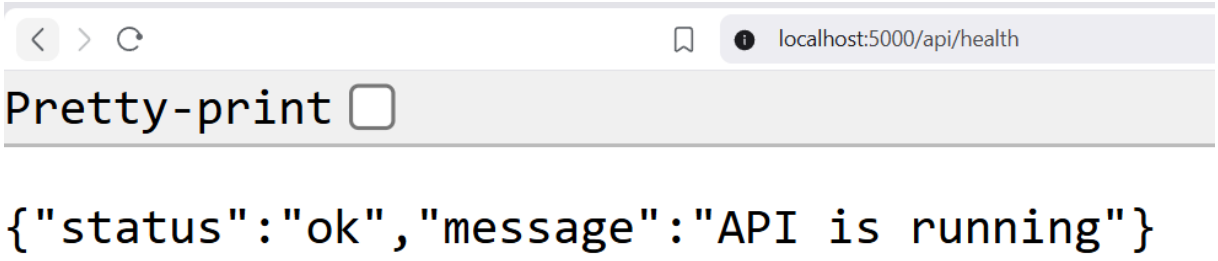

Figure 3: Test local de l'API Expense Splitter

2.5. Conteneurisation avec Docker

La conteneurisation permet d'encapsuler l'application avec toutes ses dépendances dans une image Docker optimisée :

- Dockerfile multi-stage pour réduire la taille de l'image et sécuriser l'environnement
- Construction de l'image locale : docker build -t expense-splitter:latest .

```
PS C:\Users\BADR\Documents\GitHub\Expense-Splitter> docker build -t expense-splitter .
[+] Building 1.8s (12/12) FINISHED                                          docker:desktop-linux
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 326B                                                          0.0s
 => [internal] load metadata for docker.io/library/node:18-alpine                             0.8s
 => [auth] library/node:pull token for registry-1.docker.io                                   0.0s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 129B                                                             0.0s
 => [internal] load build context                                                             0.3s
 => => transferring context: 3.60MB                                                           0.3s
 => [builder 1/4] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588  0.0s
 => => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9  0.0s
 => CACHED [builder 2/4] WORKDIR /usr/src/app                                                 0.0s
 => CACHED [builder 3/4] COPY package*.json ./                                                0.0s
 => CACHED [builder 4/4] RUN npm ci --only=production                                         0.0s
 => CACHED [stage-1 3/4] COPY --from=builder /usr/src/app/node_modules ./node_modules         0.0s
 => [stage-1 4/4] COPY . .                                                                    0.0s
 => exporting to image                                                                        0.4s
 => => exporting layers                                                                       0.2s
 => => exporting manifest sha256:d86324f65e0b5d6e6ae29075bc13b1769c140571358172229aabe28c11c3f6b0  0.0s
 => => exporting config sha256:c694276c5be97f23d177e08153e81a3fcb70569b2ef953ee311eb180c18fecde  0.0s
 => => exporting attestation manifest sha256:8ab984659d77ff7ecf96fbe06a929e7f98b85c105ea762b89de4c3c1e40c16  0.0s
 => => exporting manifest list sha256:a00caec44deccb6ee07a3600ee811361ec2d712b265da2c432b168a81502f7e8  0.0s
 => => naming to docker.io/library/expense-splitter:latest                                    0.0s
 => => unpacking to docker.io/library/expense-splitter:latest                                 0.1s
PS C:\Users\BADR\Documents\GitHub\Expense-Splitter>
```

Figure 4: Construction de l'image Docker

- Push de l'image sur Docker Hub : docker push <utilisateur>/expense-splitter:latest

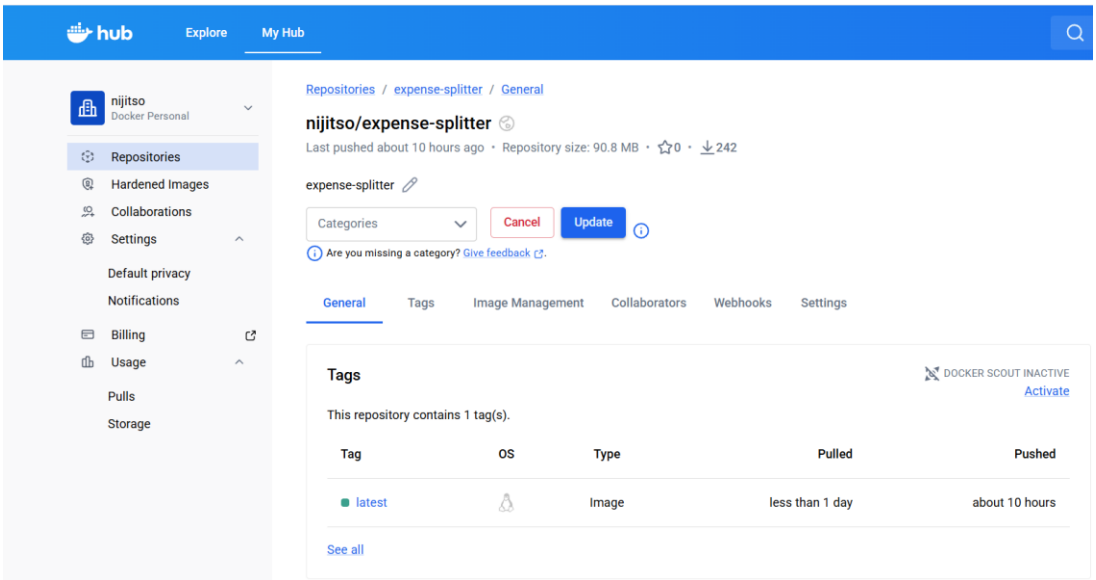

Figure 5: Image Docker publiée sur Docker Hub

Cette approche assure que l'application est reproductible et portable sur n'importe quel cluster Kubernetes.

2.6. Introduction à l'Infrastructure as Code (IaC)

Pour automatiser le provisioning et la configuration de l'infrastructure cloud, une approche Infrastructure as Code (IaC) a été adoptée. Elle permet de gérer et de provisionner les ressources à l'aide de fichiers de configuration plutôt que manuellement, garantissant ainsi reproductibilité, cohérence et rapidité de déploiement.

2.7. Provisioning avec Terraform

Terraform a été utilisé pour créer automatiquement toutes les ressources Azure nécessaires :

- Groupe de ressources, réseau virtuel, Network Security Group (NSG)
- Deux machines virtuelles Linux : nœud master et nœud worker Kubernetes
- Ouverture des ports essentiels : **22** (SSH), **80** (HTTP), **6443** (API Kubernetes), **30080** (NodePort application)

Les fichiers principaux (`main.tf`, `variables.tf` et `terraform.tfstate`) permettent de gérer la configuration déclarative et le suivi des changements dans l'infrastructure.

```
No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

master_private_ip = "10.0.1.4"
master_public_ip = "20.107.200.206"
worker_public_ip = "52.236.51.201"
PS C:\Users\BADR\Documents\GitHub\Expense-Splitter\terraform>
```

Figure 6: Provisioning de l'infrastructure avec Terraform

2.8. Configuration du cluster avec Ansible

Après le provisioning, Ansible a été utilisé pour automatiser l'installation du cluster K3s :

- Installation des dépendances nécessaires sur les nœuds (Docker, Python, etc.)
- Déploiement du serveur K3s sur le nœud master
- Ajout automatique du nœud worker au cluster via le token généré par le master
- Gestion centralisée de l'inventaire et configuration idempotente
- Cette méthode réduit les erreurs humaines et garantit une infrastructure cohérente et rapidement déployable.

```
PS C:\Users\BADR\Documents\GitHub\Expense-Splitter> wsl -d Ubuntu
badr@BADR:/mnt/c/Users/BADR/Documents/GitHub/Expense-Splitter$ cd ansible
badr@BADR:/mnt/c/Users/BADR/Documents/GitHub/Expense-Splitter/ansible$ ansible-playbook -i inventory.ini playbook.yml --check

PLAY [Configuration du Master K3s] *******************************************************************

*******************************
skipping: [20.107.200.206]

TASK [Stocker le Node Token] *************************************************************************
ok: [20.107.200.206]

TASK [Ajouter le token dans un hote virtuel] *********************************************************
changed: [20.107.200.206]

PLAY [Configuration du Worker K3s] *******************************************************************

TASK [Gathering Facts] *******************************************************************************
ok: [52.236.51.201]

TASK [Installer Docker] ******************************************************************************
ok: [52.236.51.201]

TASK [Installer K3s (Worker)] ************************************************************************
ok: [52.236.51.201]

PLAY RECAP *******************************************************************************************
20.107.200.206             : ok=5    changed=1    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
52.236.51.201              : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

badr@BADR:/mnt/c/Users/BADR/Documents/GitHub/Expense-Splitter/ansible$
```

Figure 7: Configuration automatique du cluster avec Ansible

2.9. Conclusion

Dans ce chapitre, nous avons présenté le développement de l'application Expense Splitter, sa conteneurisation avec Docker et la mise en place de l'Infrastructure as Code avec Terraform et Ansible. L'approche combinée de ces outils assure un déploiement automatisé, reproductible et maintenable. Le chapitre suivant abordera l'orchestration des conteneurs avec Kubernetes ainsi que la mise en place de la chaîne CI/CD pour un déploiement continu.

Chapitre 3 : Développement, Conteneurisation et Déploiement Automatisé

3.1. Développement de l'application

L'application Expense Splitter est une API REST en Node.js/Express avec une base MongoDB. Elle permet la gestion des utilisateurs et des groupes, la création et la modification des dépenses, le calcul automatique des soldes et balances, ainsi que la vérification de l'état du service via `/api/health`.

3.2. Conteneurisation avec Docker

L'application est empaquetée dans des conteneurs Docker pour assurer portabilité et isolation. Le processus inclut :

- Rédaction et optimisation du Dockerfile.
- Tests locaux du conteneur pour valider son fonctionnement.
- Publication de l'image sur Docker Hub pour faciliter son déploiement.

3.3. Déploiement sur Kubernetes (K3s)

Le déploiement sur Kubernetes est réalisé via des manifests YAML :

- Déploiement de MongoDB avec un volume temporaire pour le stockage.
- Déploiement de l'application Node.js avec plusieurs réplicas pour assurer la haute disponibilité.
- Configuration des variables d'environnement nécessaires à l'application.

Un service de type NodePort expose l'application sur le port 30080, permettant l'accès externe via l'adresse IP du cluster.

3.4. Pipeline CI/CD avec GitHub Actions

Une chaîne CI/CD automatisée déclenche à chaque modification du code :

- Installation des dépendances et exécution des tests.
- Construction de l'image Docker et push vers Docker Hub.
- Application automatique des manifests Kubernetes pour mettre à jour le déploiement.

Cette automatisation réduit les erreurs humaines, accélère le cycle de développement et garantit un déploiement fiable et continu.

3.5. Validation du déploiement

Après le déploiement :

- L'état des pods est vérifié pour confirmer le bon fonctionnement des conteneurs.
- L'API est testée via `/api/health`.
- Le pipeline CI/CD est validé pour s'assurer que toutes les étapes se sont correctement exécutées.

3.1. Conclusion

Ce chapitre présente la mise en œuvre pratique du développement, de la conteneurisation et du déploiement automatisé de l'application Expense Splitter. L'approche combinée Docker/Kubernetes et le pipeline CI/CD permettent un déploiement rapide, fiable et reproductible, assurant la continuité du service et la facilité d'administration de l'application.

Chapitre 4 : Orchestration Avancée et Pipeline CI/CD

4.1. Introduction

Le déploiement d'applications cloud-native nécessite non seulement une infrastructure automatisée, mais également une orchestration fine des conteneurs et un pipeline CI/CD performant pour garantir la disponibilité, la scalabilité et la fiabilité des services. Ce chapitre se concentre sur l'orchestration de l'application Expense Splitter avec Kubernetes et la mise en place du pipeline d'intégration et de déploiement continu.

4.2. Déploiement et configuration avancée avec Kubernetes

Pour assurer une gestion optimale des conteneurs, le cluster K3s est configuré de manière à séparer clairement les responsabilités entre le nœud master, chargé de l'orchestration et de la planification des tâches, et le nœud worker, responsable de l'exécution des pods applicatifs. Les manifests Kubernetes sont utilisés pour décrire l'ensemble des ressources nécessaires au fonctionnement de l'application :

- **Deployment** : définit le nombre de réplicas, la stratégie de mise à jour et l'auto-scaling pour maintenir la disponibilité.
- **Service** : permet d'exposer l'application à l'extérieur du cluster via NodePort ou LoadBalancer.
- **ConfigMaps et Secrets** : gèrent les variables d'environnement et les informations sensibles de manière sécurisée, garantissant la portabilité et la cohérence de la configuration entre les environnements.

Cette approche assure que chaque modification de l'application est automatiquement reflétée dans le cluster, avec un minimum d'intervention humaine et une haute résilience en cas de défaillance de pods.

```
Last login: Mon Mar 23 10:04:11 2026 from 41.92.103.140
azureuser@k3s-master:~$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
expense-app-b7b864754-9fzr4   1/1     Running   0          10h
expense-app-b7b864754-rlcdh   1/1     Running   0          10h
mongo-7bcb56ff4f-kd6ds        1/1     Running   0          4d4h
azureuser@k3s-master:~$
```

Figure 8: État des pods Kubernetes

4.3. Exposition des services Kubernetes

L'accès externe à l'application est configuré via un Service de type NodePort. Ce service redirige les requêtes depuis un port spécifique de la machine virtuelle vers le port interne du conteneur exécutant l'application Node.js. Cette configuration permet aux utilisateurs et aux outils de test d'accéder facilement à l'API de l'application, garantissant une validation rapide et un contrôle constant de la disponibilité du service.

```
azureuser@k3s-master:~$ kubectl get svc
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
expense-app-service  NodePort    10.43.161.144   <none>        80:30080/TCP   4d5h
kubernetes           ClusterIP   10.43.0.1       <none>        443/TCP        4d5h
mongo-service        ClusterIP   10.43.110.57    <none>        27017/TCP      4d5h
azureuser@k3s-master:~$
```

Figure 9: Exposition du service via NodePort

4.4. Pipeline CI/CD automatisé

La mise en place d'un pipeline CI/CD complet permet de réduire les erreurs humaines et d'accélérer le cycle de développement et de déploiement. Chaque push sur la branche principale du dépôt GitHub déclenche automatiquement les étapes suivantes :

1. Installation des dépendances et exécution des tests pour vérifier l'intégrité du code.

2. Construction de l'image Docker de l'application.

3. Push de l'image sur Docker Hub, assurant la disponibilité pour le déploiement.

4. Déploiement automatique sur le cluster Kubernetes via une connexion sécurisée SSH, appliquant les manifests pour mettre à jour les pods et services.

Ce pipeline garantit une mise en production continue et reproductible, tout en maintenant la cohérence et la stabilité du cluster.

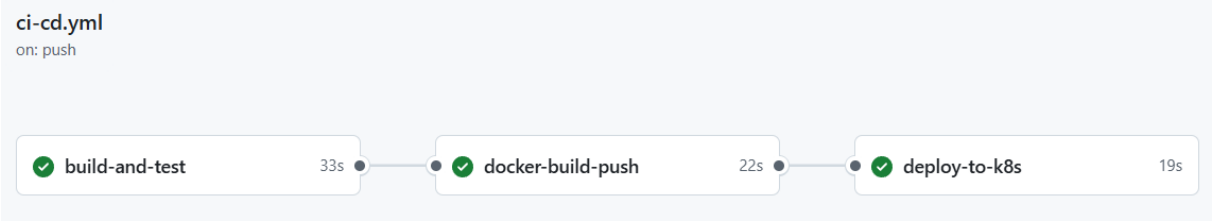

Figure 10: Exécution du pipeline CI/CD sur GitHub Actions

4.5. Validation du déploiement

Après chaque déploiement, plusieurs vérifications sont réalisées :

- Contrôle de l'état des pods avec `kubectl get pods` pour s'assurer que tous les conteneurs sont en cours d'exécution.
- Tests fonctionnels via la route **/api/health** pour vérifier que l'API est opérationnelle.
- Supervision rapide du cluster et des services pour détecter d'éventuelles anomalies ou interruptions.

Cette validation systématique permet de confirmer la fiabilité de la chaîne CI/CD et de l'orchestration Kubernetes, tout en assurant une expérience utilisateur stable et continue.

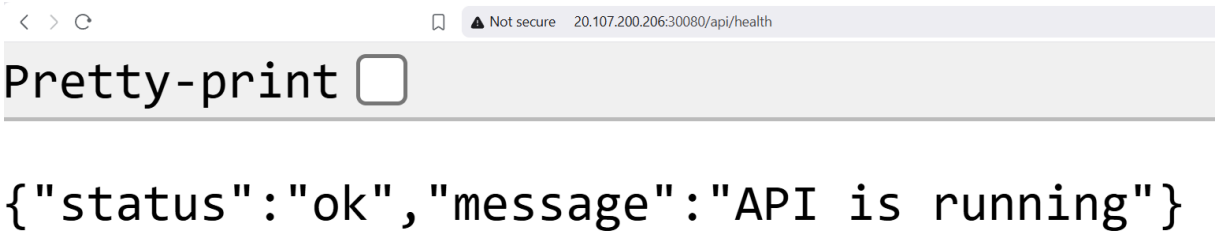

Figure 11: Accès à l'application déployée

4.6. Conclusion

Ce chapitre a présenté l'orchestration avancée de l'application Expense Splitter et la mise en place d'un pipeline CI/CD automatisé. L'utilisation de manifests Kubernetes pour gérer les déploiements, les services et les configurations sensibles, combinée à un pipeline GitHub Actions complet, permet de garantir un déploiement rapide, fiable et reproductible, tout en assurant la scalabilité et la disponibilité du service. Cette orchestration avancée constitue un pilier essentiel de l'approche DevOps adoptée dans ce projet, facilitant la maintenance et l'évolution continue de l'application.

Chapitre 5 : Résultats, analyse critique et perspectives

5.1. Validation de l'infrastructure

L'infrastructure automatisée déployée avec Terraform et Ansible a été validée par l'exécution des scripts de provisioning et de configuration. Les machines virtuelles Azure ont été créées avec succès, les nœuds Kubernetes K3s configurés et le réseau sécurisé par les NSG. L'état du cluster a été vérifié via `kubectl get nodes` pour confirmer la disponibilité des nœuds master et worker.

5.2. Tests de fonctionnement

L'application Expense Splitter a été testée sur le cluster déployé :

- Les pods ont démarré correctement et les réplicas ont été répartis sur les nœuds.
- L'accès aux services via NodePort a permis de valider l'API REST (routes principales et health check).
- Les métriques et logs collectés par Prometheus et Loki ont confirmé la stabilité du système.

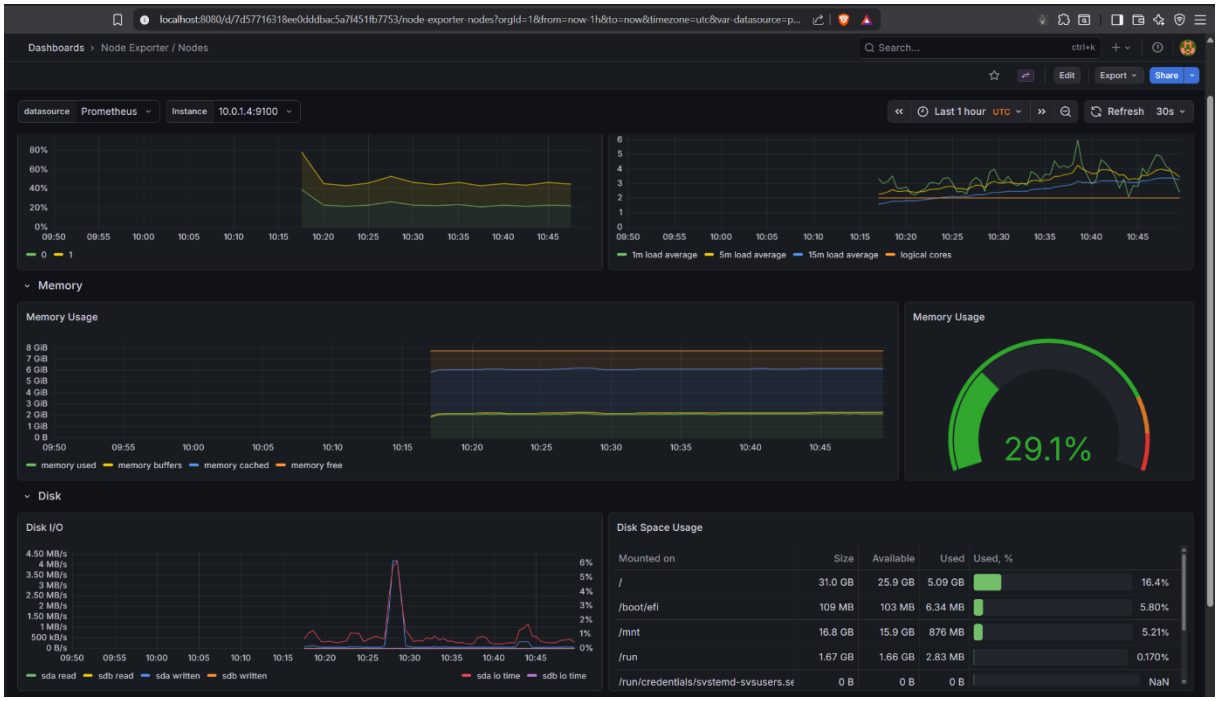

Figure 12: Dashboard Grafana pour le monitoring

5.3. Difficultés rencontrées

Parmi les principaux défis :

- Gestion des versions de K3s et compatibilité avec Docker et Helm.
- Problèmes de connexion réseau et de ports lors de l'exposition des services Kubernetes.
- Synchronisation des secrets et variables d'environnement dans le pipeline CI/CD.

5.4. Solutions apportées

- Mise à jour des playbooks Ansible et ajustement des manifests Kubernetes pour résoudre les conflits de ports.
- Centralisation des secrets via GitHub Actions Secrets et ConfigMaps Kubernetes pour sécuriser les credentials.
- Tests itératifs et vérifications via `kubectl logs` et dashboards Grafana pour confirmer la bonne exécution des services.

5.5. Analyse critique du projet

Points forts :

- Infrastructure automatisée, reproductible et scalable.
- Pipeline CI/CD efficace, réduisant l'intervention manuelle.
- Monitoring et observabilité en temps réel grâce à Prometheus, Grafana et Loki.

Limites :

- Cluster K3s limité en termes de fonctionnalités avancées par rapport à un cluster Kubernetes complet.
- Déploiement multi-environnement non encore automatisé.
- Sécurité des applications et du cluster perfectible (ex: chiffrement des communications inter-pods).

5.6. Comparaison technique

- **K3s vs Kubeadm :** K3s plus léger et rapide à déployer, mais certaines fonctionnalités avancées sont absentes.

- **Docker vs VM classique :** Conteneurs plus rapides, isolés et portables, alors que les VM sont plus lourdes et moins flexibles.
- **CI/CD manuel vs automatisé :** Automatisation via GitHub Actions réduit les erreurs humaines et accélère le cycle de déploiement.

5.7. Perspectives d'amélioration

- **Monitoring avancé :** Intégration d'alerting et dashboards supplémentaires.
- **Sécurité :** Mise en place de certificats TLS, secrets chiffrés, RBAC renforcé et éventuellement Service Mesh comme Istio.
- **Autoscaling :** Activer le scaling automatique des pods selon la charge de travail.
- **Multi-environment deployment :** Déploiement automatisé pour staging, test et production avec des pipelines distincts.

5.8. Conclusion

Le chapitre a permis de valider l'infrastructure et le fonctionnement de l'application Expense Splitter sur Kubernetes, de constater les points forts du projet comme l'automatisation et la reproductibilité, d'identifier ses limites (sécurité, multi-environnement) et de proposer des perspectives d'amélioration pour renforcer la scalabilité, la supervision et la robustesse du déploiement

Conclusion

La réalisation de ce projet avait pour objectif principal de mettre en place une solution DevOps complète et automatisée pour l'application Expense Splitter, depuis le développement local jusqu'au déploiement sur un cluster Kubernetes cloud, en intégrant conteneurisation, Infrastructure as Code, CI/CD et monitoring. Les résultats obtenus confirment la réussite de ces objectifs : l'application est déployée de manière reproductible, sécurisée et observable, avec un pipeline CI/CD fonctionnel et un cluster K3s opérationnel. Sur le plan technique, ce projet a permis d'approfondir la maîtrise de Terraform, Ansible, Docker, Kubernetes, et des outils de monitoring comme Prometheus et Grafana. Les compétences acquises incluent la gestion d'infrastructures cloud automatisées, l'orchestration de conteneurs, l'intégration continue et le déploiement continu, ainsi que la supervision des services applicatifs. Enfin, ce projet ouvre la voie à des améliorations futures telles que la mise en place de l'autoscaling, l'optimisation de la sécurité, le déploiement multi-environnements et l'intégration de solutions avancées comme un service mesh pour une meilleure résilience des applications cloud-native.